

Faculty of Engineering

Topics in Computer Science

POMDP

PhD. Willy Ugarte Rojas
willyugarte@gmail.com

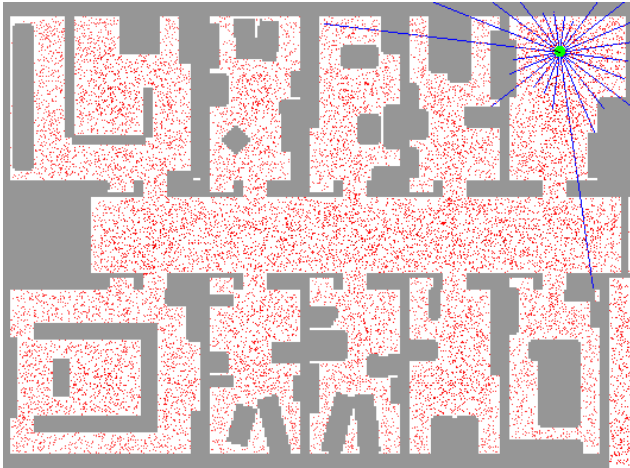


UPC

Universidad Peruana
de ciencias Aplicadas

Sequential decision-making under uncertainty

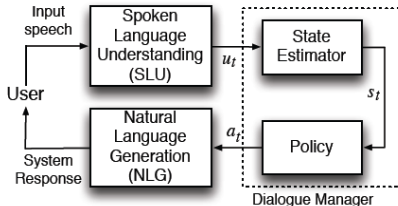
Sequential decision-making under uncertainty



http://www.cs.washington.edu/ai/Mobile_Robotics/mcl/animations/global-floor.gif

Sequential decision-making under uncertainty

<http://mi.eng.cam.ac.uk/~sjy/papers/ygtw13.pdf>



Observation	True State	Belief Entropy	Action	Reward
flo hello	request_begun	0.406	say_hello	100
flo what is like	start_meds	2.735	ask_repeat	-100
flo what time is it for will the	want_time	0.490	say_time	100
flo was on abc	want_tv	1.176	ask_which_station	-1
flo was on abc	want_abc	0.886	say_abc	100
flo what is on nbc	want_nbc	1.375	confirm_channel_nbc	-1
flo yes	want_nbc	0.062	say_nbc	100
flo go to the that pretty good what	send_robot	0.864	ask_robot_where	-1
flo that that hello be	send_robot_bedroom	1.839	confirm_robot_place	-1
flo the bedroom any i	send_robot_bedroom	0.194	go_to_bedroom	100
flo go it eight a hello	send_robot	1.110	ask_robot_where	-1
flo the kitchen hello	send_robot_kitchen	1.184	go_to_kitchen	100

<http://www.cs.cmu.edu/nursebot>

Markov models

Markov models



Markov model = stochastic model that assumes Markov property.

- **stochastic model**: models process where state depends on previous states in non-deterministic way.
- **Markov property**: the probability distribution of future states, *conditioned on both past and present values*, depends only upon present state:

“given the present, the future does not depend on the past”

Generally, this assumption enables reasoning and computation with the model that would otherwise be intractable.

Markov model types



	Prediction	Planning
Fully observable	Markov chain	MDP (Markov decision process)
Partially observable	Hidden Markov model	POMDP (Partially observable Markov decision process)

Typically for optimisation purposes

Prediction models can be represented at variable level by a (Dynamic) Bayesian network:



MDPs: outline



- Search in **non-deterministic** environments
- Solution: **optimal policy (plan) of actions that maximizes rewards**

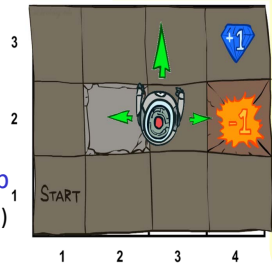
(decision-theoretic planning)

- Algorithm:
 - Bellman equation
 - value iteration
- Link with learning

Running example: Grid World



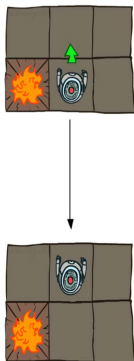
- A maze-like problem
 - The agent lives in a grid, where walls block the agent's path
- *Noisy* movement: actions do not always go as planned
 - If wall in chosen direction, then stay put;
 - 80% of the time, the action North takes the agent North
 - 10% of the time, North takes the agent West; 10% East (same deviation for other actions)
- The agent receives rewards each “time” step
 - Small “living” reward each step (can be negative)
 - Big rewards come at the end (good or bad)
- **Goal:** maximize sum of rewards



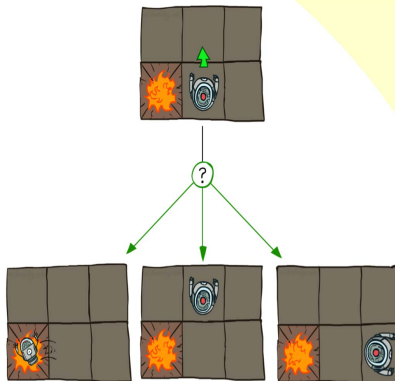
Search in non-deterministic environments: Grid World example



Deterministic Grid World



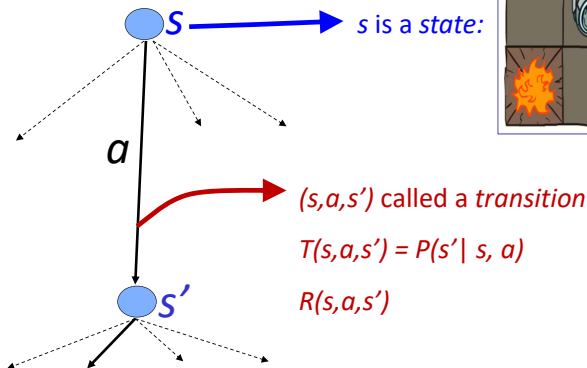
Stochastic Grid World
Noisy movement: actions do not always go as planned



MDP: in search of 'plan' that maximises reward



Each MDP state projects a search tree



Goals, rewards and optimality criteria



- Planning goals: encoded in reward function

Example: achieving a state satisfying property P at minimal cost is encoded by making any state satisfying P a zero-reward absorbing state, and assigning all other states negative reward.

- Rewards: additive and time-separable
- Transitions: effect is uncertain
- Objective: maximize expected total reward;
future rewards may be discounted
- Planning horizon: finite, infinite or indefinite (latter: special case of infinite; guaranteed to reach terminal state)

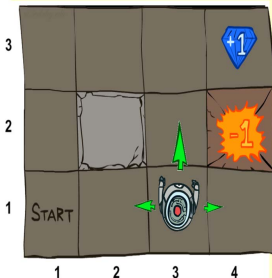
Markov Decision Processes



MDPs are non-deterministic search problems

An MDP is defined by:

- A **set of states** $s \in S$
- A **set of actions** $a \in A$
- A **transition function** $T(s, a, s')$
 - Probability that a from s leads to s' ,
i.e., $P(s' | s, a)$
 - Also called the **model** or the **dynamics**
- A **reward function** $R(s, a, s')$
 - Sometimes just $R(s)$ or $R(s')$
- A **start state**
- Sometimes a **terminal state**



Discounting



1

Worth
Now



γ

Worth Next
Step



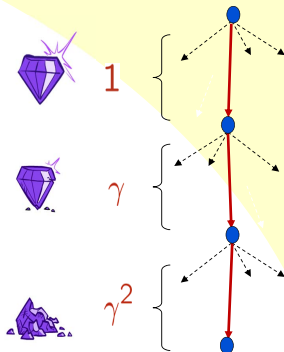
γ^2

Worth In Two
Steps

Discounting: implementation



- How to discount?
 - Each time we descend a level, we multiply in the discount once
- Why discount?
 - Sooner rewards probably do have higher utility than later rewards
 - Also helps our algorithms **converge**
- Example:
 - Value of receiving [1,2,3] with discount of 0.5 = $1*1 + 0.5*2 + 0.25*3$
 - Which is less than that of [3,2,1]



Return: reward in the long run



- **Episodic tasks:** interaction breaks naturally into episodes, e.g., plays of a game, trips through a maze.

➔ return is total reward from 'time' t to 'time' T , ending an episode:

$$R_t = r_{t+1} + r_{t+2} + \dots + r_T = \sum_{k=0}^{T-t-1} r_{t+k+1}$$

- **Continuing tasks:** interaction does not have natural episodes.

➔ return **discounted** by **discount rate** γ , $0 \leq \gamma \leq 1$:

$$R_t = r_{t+1} + \gamma \cdot r_{t+2} + \gamma^2 \cdot r_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k \cdot r_{t+k+1}$$

(shortsighted $0 \leftarrow \gamma \rightarrow 1$ farsighted)

Solving MDPs

Value Iteration (VI)



'Tree backup': define $V_k(s)$ as the optimal value of s still to be obtained if 'the game' ends in k more time steps

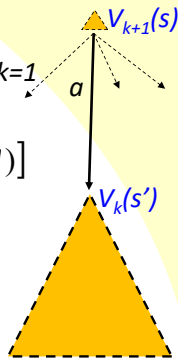
- Start with $V_0(s) = 0$ for **all** s (including terminal s)
- *Terminal state rewards (if any) typically added at $k=1$*
- Given $V_k(s)$ values, compute for each s

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')] \\ (= \max_a Q_k(s, a))$$

- Repeat until convergence of V -values

Theorem: VI will converge to unique optimal values

- Basic idea: approximations get refined towards optimal values



Policy Iteration



- Alternative approach for optimal values:
 - Step 1: Policy evaluation: calculate returns for some fixed policy until convergence
 - Step 2: Policy improvement: update policy using one-step look-ahead with resulting converged (but not optimal!) returns as future values
 - Repeat steps until policy converges
- This is policy iteration
 - It's still optimal!
 - Can converge (much) faster under some conditions

Comparison VI and PI

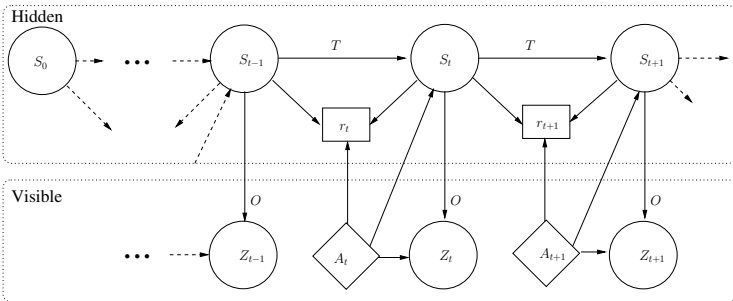


- Both are DP algorithms for solving MDPs
- both compute the same thing (all optimal values)
- In value iteration:
 - Every iteration updates both values and, implicitly, policy
 - Doesn't track policy: taking max over actions implicitly recomputes it
- In policy iteration:
 - Do several passes that update returns with fixed policy (each pass is fast: we consider only one action, not all)
 - After the policy is evaluated, a new policy is chosen (slow like a value iteration pass)
 - The new policy will be better (or we're done)

POMDP

Partially Observable MDP

- POMDP defined by n-tuple $\langle S, A, Z, T, O, R \rangle$,
where $\langle S, A, T, R \rangle$ are same as in an MDP.
- States are hidden $\rightarrow$ Observations (Z)
- Observation function $O(s,a,z) := Pr(z \mid s, a)$

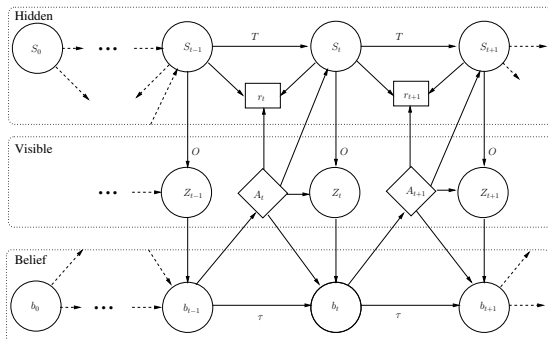


Belief-MDP (Åström, 1965)

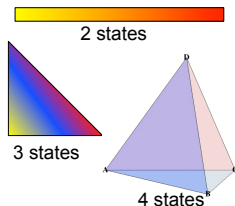
- Belief-state, b_t : Probability distribution over states, is a sufficient statistic of **history** $\{a_0, z_0, \dots, a_t, z_t\}$.



Karl Åström



The belief simplex

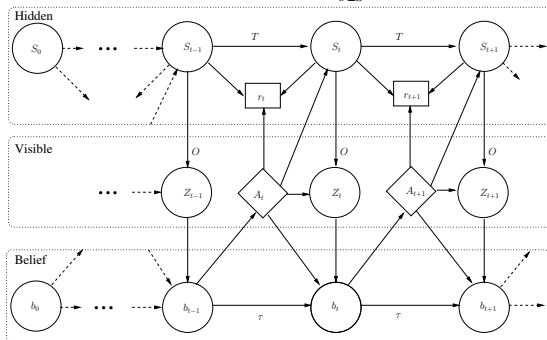


Belief-MDP (Åström, 1965)

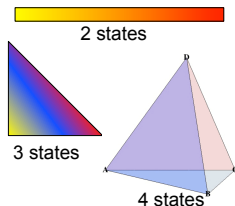
- **Belief MDP:** $\langle S, A, Z, T, O, R \rangle \rightarrow \langle \Delta, A, \tau, \rho, b_0 \rangle$
- **Transition function:** $\tau(b, a, b')$
- **Expected reward:** $\rho(b, a) = \sum_{s \in S} b(s) R(s, a)$



Karl Åström



The belief simplex

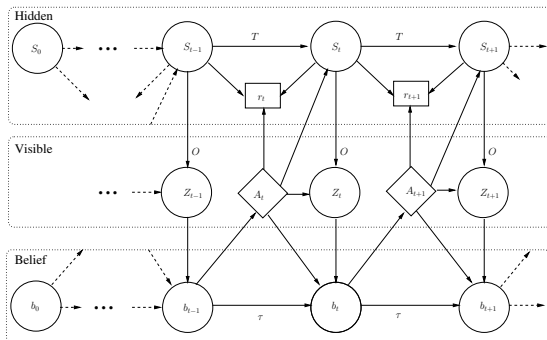


Belief-MDP (Astrom, 1965)

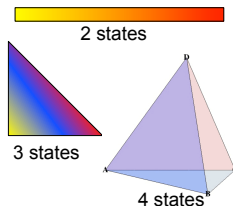
- Belief update: Bayes Rule!
- Value fn: Bellman's equation!
- Policy: $\pi : \mathbf{b} \longrightarrow \mathbf{a}$



Karl Åström



The belief simplex

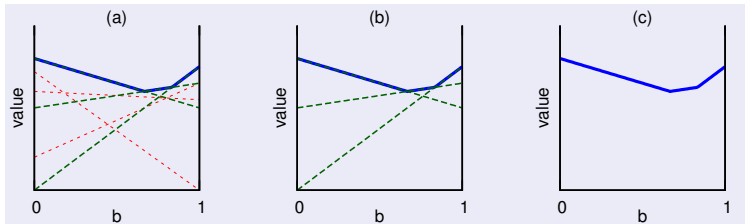


Early POMDP solution methods

- Observe: Reward function is linear $\rho(b,a) = \sum_{s \in S} b(s)R(s,a)$
- $V_t^*(b)$ is therefore piecewise linear and convex.
- Set of hyper-planes, named α -vectors, represent value function $V(b) = \max_{\alpha \in I^*} \alpha \cdot b$



Edward Sondik



Early POMDP solution methods

- Observe: Reward function is linear $\rho(b,a) = \sum_{s \in S} b(s)R(s,a)$
- $V_t^*(b)$ is therefore piecewise linear and convex.
- Set of hyper-planes, named α -vectors, represent value function $V(b) = \max_{\alpha \in \Gamma} \alpha \cdot b$



Edward Sondik

Exact Solution Methods

Propagate, combine and prune hyperplanes using the Bellman Equation



(Monahan, 1982)
Batch Enumeration



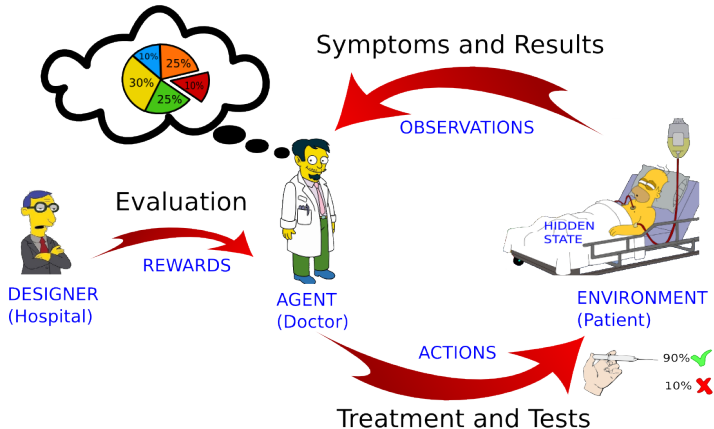
(Littman, 1996)
Witness Algorithm



(Cassandra, 1998)
Incremental Pruning

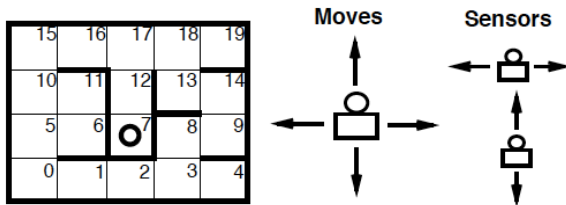
Problem: Exact solving POMDPs is **PSPACE-Complete**

Solving small information-gathering problems



From Mauricio Araya-Lopez, JFPDA 2013.

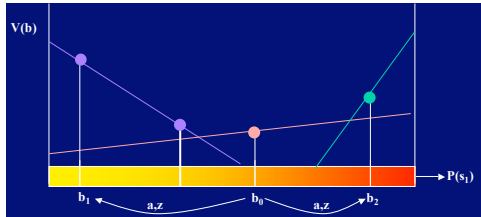
Gridworld POMDPs – Solved!



<http://people.cs.pitt.edu/~milos/research/JAIR-2000.pdf>

Point-based value iteration (Pineau et al., 2003)

- Select a small set of reachable belief points.
- Perform Bellman updates at those points, keeping value & gradient.
- Anytime and polynomial algorithm.
- Bounded error depends on density of points.



Other Point-based Algorithms

PERSEUS



(Spaan and Vlassis, 2005)

HSVI



(Smith and Simmons, 2005)

FSVI



(Shani et al., 2007)

SARSOP



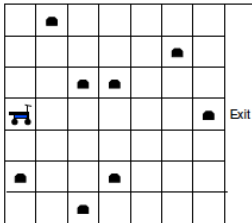
(Kumiawati et al., 2008)

GapMin

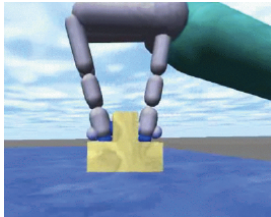


(Poupert et al., 2011)

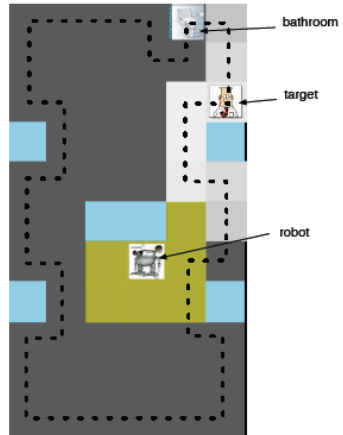
POMDPs with thousands of states – SOLVED!



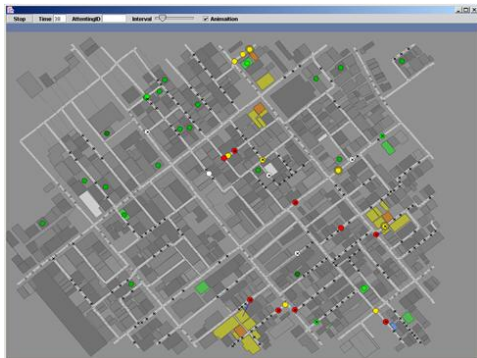
http://www.cs.cmu.edu/~trey/papers/smith04_hsvi.pdf



<http://bigbird.comp.nus.edu.sg/~hannakur/publications.html>



Harder problem: Robocup Rescue Simulation



From Sébastien Paquet et al., 2005

Highly dynamic environment.

Approx. 30 agents of 6 types

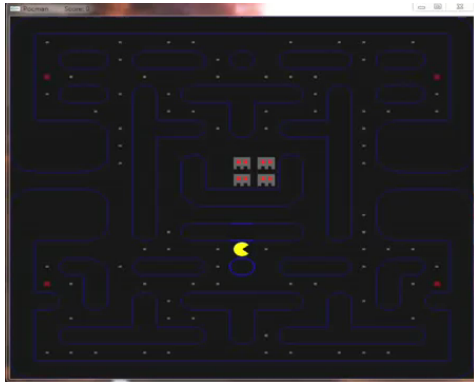
Partially observable state.

Real-time constraints on
agents' response time.

Agents face unknown instances
of the environment.

Are we done yet?

- Pacman problem: $|S|=10^{56}$, $|A|=4$, $|Z|=1024$.



<http://www0.cs.ucl.ac.uk/staff/D.Silver/web/Applications.html>

There is no simulator for this!



<http://www.tech.plym.ac.uk/SoCCE/CRNS/staff/fbroz/>



<http://web.mit.edu/nickroy/www/research.html>



CONCLUSION

- What if the model is not known?
- There are currently very few RL methods that can be used (multiagent, partially observable)
 - Model-free gradient-based policy search methods [Dutech et al., IJCAI 01](#); [Peshkin et al., UAI 00](#)
 - Learning using local signals and modeling the remaining agents as noise [Chang et al., NIPS 04](#)
 - Communication and sample-based planning to generate best-response policies [Banerjee et al., AAAI 12](#)
 - Learning from demonstrations [Liu et al., IJCAI 15](#), [Liu et al., AAAI 16](#)
 - Deep RL for Dec-POMDPs [Foerster et al., NIPS 16](#), [Omidshafiei et al., ICM:17](#)
- An active (but difficult) area of current research

Dec-POMDP resources

- Book:
 - <http://www.springer.com/us/book/9783319289274>
- Websites:
 - <http://masplan.org/>
 - <http://rbr.cs.umass.edu/decpomdp/>
- We're working on getting more online

